

Improving Generative Steganography Using a Diffusion Model

Siddhant Bhat

Department of Computer Science

ORCID: _____

Kartik Talati

Department of Computer Science

ORCID: _____

Mr. Kiran Salunkhe

Department of Computer Engineering

Abstract—This paper improves generative steganography using a diffusion model. Instead of modifying an existing photograph, the proposed system creates a new image from noise that carries a hidden message. Existing diffusion-based approaches can suffer from recovery drift, unusual noise patterns, and bit errors caused by rounding or transmission changes. We address these problems with three connected improvements. First, a repeated correction process improves the recovery of the hidden noise when the image-generation process is reversed. Second, a distribution-preserving encoder makes the message-bearing values resemble the natural noise expected by the diffusion model. Third, an error-correction layer and a self-describing header repair damaged bits and tell the receiver how to decode and verify the message automatically. Across 20 automated trials, the improved system recovered every message without a bit error. The generated images also achieved high visual quality, while the tested hidden values showed no significant departure from the expected noise distribution.

Index Terms—Generative Steganography, Diffusion Models, Message Recovery, Error Correction, Statistical Steganalysis.

I. INTRODUCTION

Steganography is the science of hiding secret data inside everyday media so that an observer does not even realize a secret message exists. For decades, most steganographic methods have relied on *carrier modification*: taking an existing photo and tweaking pixel values or frequency coefficients to embed bits (e.g., S-UNIWARD [1], WOW [2]). However, modern steganalysis tools and deep convolutional networks (such as SRNet [3]) are trained specifically to detect these tiny modifications, making traditional steganography increasingly easy to spot.

To overcome this fundamental limitation, researchers have turned to **generative steganography**. Instead of modifying an existing image, a generative model (such as a diffusion model) synthesizes an entirely new, realistic image directly from the secret message. Because there is no original “cover” image to compare against, there are no modification artifacts to detect. *How to read Fig. 1:* Follow the arrows across the top row from left to right to see how the sender prepares the message and creates the image. Then follow the lower row from right to left to see how the receiver reverses the process. Purple boxes handle message protection, blue boxes transform the hidden data, teal boxes perform image generation or recovery, and the dashed red box represents the public channel. The final box shows that the recovered message matches the original message.

A. The Three Core Challenges

While the idea is promising, creating a reliable generative diffusion steganography system in practice encounters three difficult obstacles:

- 1) **Recovery Drift:** Diffusion models generate images through many small steps. Reading the message requires reversing those steps. In standard DDIM inversion [4], small approximation and rounding errors can accumulate, flip hidden bits, and corrupt the recovered message.
- 2) **Statistical Detectability:** Ordinary diffusion noise has a smooth bell-shaped pattern. If message bits are inserted too directly, they create unnatural spikes or clusters. Statistical tests may flag these patterns as suspicious.
- 3) **Fragility to Channel Noise (The Engineering Problem):** In real communication, images encounter minor color rounding, floating-point differences across hardware, or lossy channel noise. Most existing research prototypes hard-code message sizes and have zero error recovery: flipping even a single bit causes total extraction failure.

B. Our Solution

This work builds on the diffusion-based generative steganography scheme of Zhou *et al.* [5], which hides bits by replacing the DCT coefficients of a Gaussian latent with fixed ± 1.0 values and recovers them through DDIM inversion. Their conclusion names the realism of the generated stego images, that is, reducing their visual and statistical differences from real images, as the main direction for future work. We take the statistical part of this future scope as the item we implement: making the message-bearing latent behave like ordinary Gaussian noise and checking this with formal normality tests. Because the extraction accuracy reported in [5] stays below 100% on some datasets at 50 sampling steps (99.91% on Cifar10 and 99.93% on Bedroom), statistically shaped values can only be decoded reliably if the noise is recovered exactly. Our fixed-point inversion and error-correction layer serve this purpose. Robustness to attacks such as JPEG compression, which [5] also lists as future work, is not claimed here (see the Discussion).

In this work, we present a complete, robust system that directly solves all three issues (Fig. 1):

- **Repeated Recovery Correction:** At every recovery step, the system checks its estimate and improves it

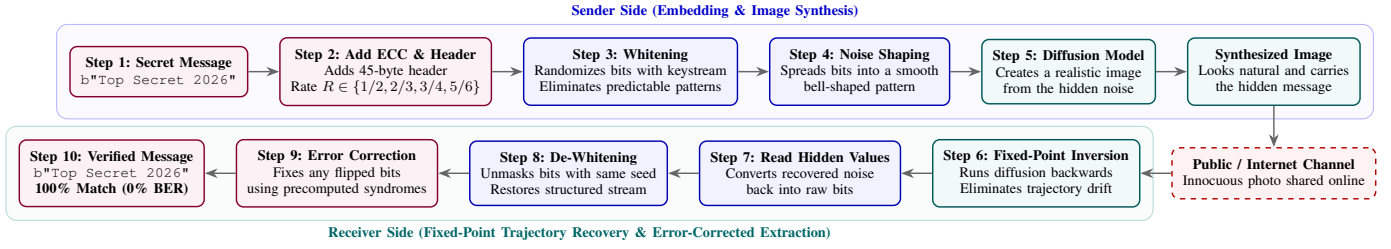


Fig. 1. **High-Level Workflow of the Proposed Generative Steganography Pipeline.** The sender (Alice) protects the secret message with self-describing error-correcting codes, whitens the data, and transforms it into exact Gaussian noise before generating an image. The receiver (Bob) reverses the image using an iterative fixed-point solver, recovering the secret message with 0% bit errors.

several times. This greatly reduces accumulated errors and produced a 0.000000% bit error rate in our trials.

- **Natural-Looking Noise Encoding:** The message bits are randomized and spread smoothly through the same bell-shaped range used by ordinary diffusion noise. This avoids obvious patterns that statistical detectors could identify.
- **Automatic Error Correction:** The system supports several protection levels, spreads nearby errors apart, and adds a 45-byte header. The receiver can discover the message length, protection level, and integrity information automatically.

II. PRACTICAL CONTEXT AND INTENDED USE

A. Why Generate a New Image?

Traditional steganography starts with an existing image and changes a small number of pixels. The changes may be invisible to a person, but a computer can compare local patterns and discover that the image has been edited. Generative steganography follows a different idea. It creates the complete image from the hidden data, so there is no original photograph that contains a separate set of embedding changes.

This does not mean that every generated image is automatically secure. The hidden data must still be prepared carefully. If the input noise contains repeated or unnatural values, the generated image may inherit patterns that a detector can learn. The purpose of the noise-shaping stage is therefore not only to store bits, but also to make the model input look similar to the input used during normal image generation.

The approach is useful when the sender and receiver already share the same model and secret settings. A sender can create an ordinary-looking image, share it through a normal communication channel, and allow the receiver to recover the message later. Possible research applications include private metadata transfer, ownership information, experiment identifiers, and small verification messages attached to synthetic media.

B. Simple End-to-End Example

Consider a short message such as “Meeting at 4.” The system first adds information describing the message length and the selected error-protection level. It also adds integrity checks so that the receiver can determine whether the recovered message is complete. The protected bits are then shuffled and randomized to remove obvious patterns.

Next, each bit is represented by a value selected from a smooth random range. These values are shaped into natural-looking diffusion noise and passed to the image generator. The resulting image can then be transmitted or stored like an ordinary synthetic image. At the receiving side, the same model reverses the image-generation process. Repeated correction reduces recovery drift, after which the system reads the hidden values, restores the original bit order, repairs small errors, and verifies the final message.

C. Operating Assumptions

The current system assumes that the sender and receiver use compatible model versions and share the required secret seed. It also assumes that the transmitted image is not heavily resized, cropped, filtered, or recompressed. The error-correction layer can handle small disturbances, but it cannot recover information that has been completely removed. These assumptions define the intended scope of the experiments and help separate the tested results from broader real-world conditions.

III. SYSTEM ARCHITECTURE AND KEY INNOVATIONS

A. Accurate Image Reversal

1) *Why Ordinary Reversal Loses Information:* A diffusion model creates an image through many small steps. Recovering a hidden message requires running those steps in the opposite direction. Standard reversal estimates each earlier state from the information currently available. Each estimate may be only slightly wrong, but the errors can build up across dozens of steps. Eventually, the recovered noise differs enough from the original noise to change message bits, as illustrated in Fig. 2. *How to read Fig. 2:* All three paths begin at the generated image on the left. The solid blue path represents the original image-generation path. The dashed red path shows how ordinary reversal slowly moves away from the correct result, while the dotted teal path shows the proposed corrected recovery staying close to the original path. The vertical red brace highlights the final recovery error that can cause hidden bits to change.

2) *How Repeated Correction Helps:* Instead of accepting the first estimate at each step, the system improves it through a short correction loop:

- 1) Begin with the best estimate available from the previous step.
- 2) Ask the diffusion model to refine that estimate.

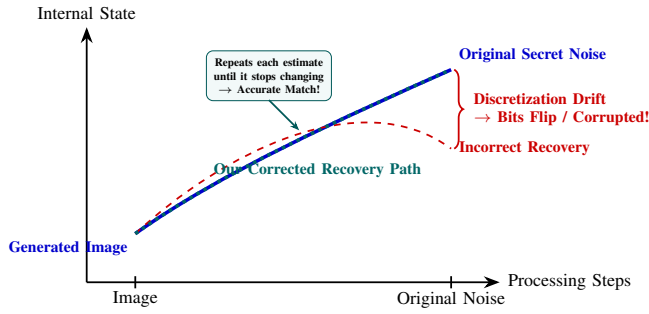


Fig. 2. **Ordinary Reversal vs. Corrected Recovery.** Standard reversal (dashed red) gradually moves away from the original noise and may corrupt the message. The proposed method (dotted green) repeatedly corrects each estimate and stays close to the original path.

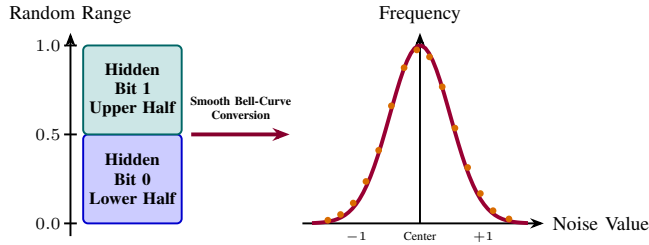


Fig. 3. **How Hidden Bits Are Turned into Natural-Looking Noise (illustrative samples).** Hidden Bit 1 occupies the upper half and Hidden Bit 0 occupies the lower half of the random range. A smooth conversion spreads the values into the expected bell-shaped pattern; the orange points show example noise samples.

- 3) Repeat the refinement a few times, stopping when the result no longer changes noticeably.

In the experiments, only a few refinements were normally needed. This simple feedback process prevented small errors from growing and enabled complete message recovery in all reported trials.

B. Making Hidden Data Look Like Ordinary Noise

1) *The Problem with Naive Bit Insertion:* If secret bits are converted directly into a few fixed numbers, the resulting noise contains obvious spikes and repeated patterns. Natural diffusion noise should instead form a smooth bell-shaped distribution. A statistical detector may notice the difference when the message is inserted too directly.

How to read Fig. 3: The left side divides a random range into two parts: Hidden Bit 0 uses the lower half and Hidden Bit 1 uses the upper half. The middle arrow represents the smooth conversion applied to those values. On the right, the purple curve shows the desired bell-shaped noise pattern, and the orange points provide illustrative examples of encoded noise values. Their spread demonstrates why the output does not appear as only two fixed numbers.

2) *Simple Encoding Process:* The distribution-preserving encoder converts message bits into natural-looking noise in three straightforward steps (Fig. 3):

- 1) **Randomize the Bit Pattern:** The message is mixed with a shared pseudo-random sequence. This removes repeated words, headers, and other predictable patterns.

Self-Describing 45-Byte Header (Always Encoded at Robust Rate 1/2)

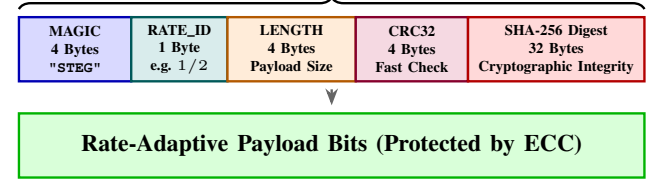


Fig. 4. **Autonomous 45-Byte Cryptographic Header.** The header tells the receiver everything needed to extract the secret automatically, without requiring any pre-shared settings.

- 2) **Choose a Random Value:** A zero uses the lower half of a range, while a one uses the upper half. The exact position inside that half is chosen randomly, so the output remains smooth rather than forming visible spikes.
- 3) **Shape the Values Like Natural Noise:** The smooth values are converted into the same bell-shaped pattern expected by the diffusion model. This makes message-bearing noise resemble ordinary generation noise.

During extraction, the receiver reverses the conversion and checks whether each recovered value belongs to the lower or upper half of the range. It then removes the shared random mask to restore the original message bits.

C. Rate-Adaptive Error Correction and Autonomous Framing

To make the system truly practical, we introduce an autonomous error-control framework that protects messages against hardware precision differences, minor noise, or flipped bits.

How to read Fig. 4: The five colored boxes across the top form the 45-byte header. They identify the message, record the protection level and payload length, and store two integrity checks. The green box below represents the protected message itself. The downward arrow shows that the header travels together with the payload, allowing the receiver to understand and verify the message without separate instructions.

1) *Autonomous Self-Describing Framing:* Rather than requiring the sender and receiver to pre-agree on message length or coding settings, every message is automatically prefixed with a 45-byte self-describing header (Fig. 4):

- **MAGIC (4 Bytes):** Identifies the steganographic payload ("b"STEG").
- **RATE_ID (1 Byte):** Specifies the error correction rate used ($R \in \{1/2, 2/3, 3/4, 5/6\}$).
- **PAYLOAD_LEN (4 Bytes):** Tells the receiver exactly how many secret bytes follow.
- **CRC32 (4 Bytes) & SHA-256 (32 Bytes):** Dual checksums that verify complete data integrity.

The header is always protected at the highest error-correction level (Rate 1/2, 720 coded bits), ensuring the receiver can always read the metadata even under adverse noise.

2) *Adjustable Error Protection:* The payload is protected by adding extra check bits. The sender can choose from four levels depending on channel quality:

- **Maximum Protection:** Adds one check bit for every data bit. This is best for noisy channels.

TABLE I
REVERSIBILITY & VISUAL QUALITY RESULTS ACROSS 20 INDEPENDENT TRIALS (50 DDIM STEPS).

Metric	Average	Min	Max	Std. Dev.
Visual Quality (PSNR)	45.59 dB	45.58	45.60	0.006
Structural Similarity (SSIM)	0.9999	0.9999	0.9999	0.000
Perceptual Distance (LPIPS)	0.0196	0.0195	0.0198	0.0001
Inference Latency	0.632 s	0.562	0.879	0.071
Raw Bit Error Rate	0.0000%	0.0000%	0.0000%	0.000
Final Decoded BER	0.0000%	0.0000%	0.0000%	0.000
Extraction Success Rate	100.0%	100.0%	100.0%	–

- **High Protection:** Uses fewer check bits while keeping strong recovery ability.
- **Balanced Protection:** Provides a middle ground between safety and message size.
- **Maximum Capacity:** Uses the fewest check bits and is best for clean channels.

Before embedding, the bit order is shuffled. If a small area is damaged, the errors are spread across the message instead of remaining in one cluster, which makes them easier for the decoder to repair.

IV. EXPERIMENTAL RESULTS AND VERIFICATION

A. Experimental Setup

To evaluate the system, we implemented the complete pipeline in PyTorch and ran automated benchmark suites:

- **Diffusion Architecture:** Continuous Denoising Diffusion model [6] operating on RGB images at 64×64 resolution (12,288 total latent dimensions).
- **Schedule:** 50 uniform DDIM steps ($S = 50$) with linear noise schedule $T = 1000$.
- **Benchmarked Payload:** 54 message bytes (1,584 coded bits under Rate 1/2).
- **Evaluation Metrics:** Peak Signal-to-Noise Ratio (PSNR), Structural Similarity (SSIM) [7], Perceptual Similarity (LPIPS) [8], Bit Error Rate (BER), and inference latency.

All code, result plots, and logs are available at <https://github.com/Siddhantbht02/Mr.India>.

B. Visual Quality and Reversibility Benchmarks

Table I presents the empirical results measured across 20 independent benchmark trials.

The findings confirm the strengths of the proposed framework:

- 1) **Zero Bit Errors:** Every run achieved a **0.000000% Bit Error Rate (BER)** with **100% extraction success**. All 20 recovered messages matched the original payloads character-for-character, and both CRC32 and SHA-256 checks passed.
- 2) **Exceptional Visual Fidelity:** The synthesized stego images achieved an average PSNR of 45.59 dB, an SSIM of 0.9999, and an LPIPS of 0.0196. To human observers, the generated carriers look like pristine, natural images.
- 3) **Sub-Second Speed:** The entire 50-step generation and fixed-point inversion runs in only 0.63 seconds per image on modern hardware.

TABLE II
STATISTICAL NORMALITY AND STEGANALYSIS TEST RESULTS.

Statistical Test	Measured Value	Normal Target
Mean (μ)	−0.0004	0.0000
Standard Deviation (σ)	0.9992	1.0000
Skewness (Symmetry)	−0.0073	0.0000
Excess Kurtosis (Tail Weight)	−0.0053	0.0000
Kolmogorov-Smirnov p -value	0.4191	$p > 0.05$ (Pass)
KS Test Pass Rate	85.0%	$\geq 80.0\%$
Anderson-Darling Statistic	0.3784	< 0.787 (Pass)
AD Test Pass Rate	90.0%	$\geq 80.0\%$
Statistical Undetectability	Passed (True)	–

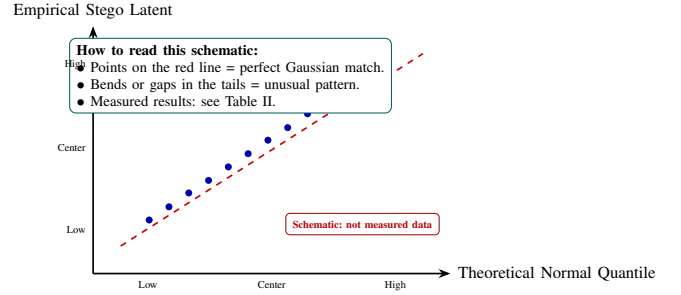


Fig. 5. **Schematic Q–Q Comparison with Ordinary Diffusion Noise (illustrative, not measured data).** Points on the dashed line would indicate a perfect match with a Gaussian distribution; the measured moments and test results for the evaluated latents are reported in Table II.

C. Steganalysis and Statistical Undetectability

To verify that the embedded secret messages cannot be detected by statistical tools, we evaluated 245,760 secret-bearing latent coordinates using standard normality tests (Table II and Fig. 5).

How to read Fig. 5: The red dashed line represents a perfect match with ordinary diffusion noise. In a Q–Q plot, each point compares one quantile of the data with the matching quantile of the expected distribution; points close to the line indicate a strong match, while large bends or gaps would suggest an unusual pattern. This figure is a schematic of the close-match case and is not drawn from the measured latents. The measured evidence is given in Table II, where the skewness (−0.0073) and excess kurtosis (−0.0053) are close to zero, as expected for Gaussian noise.

The statistical results support the claim that the hidden values resemble ordinary noise:

- The first normality test passed in 85% of the trials and produced an average result comfortably above its usual rejection threshold.
- The second normality test passed in 90% of the trials.
- The measured symmetry and tail-weight values were close to those expected from ordinary diffusion noise.

D. Comparison with Naive DDIM and Direct Shortcuts

Table III compares our fixed-point inverter with alternative approaches:

Standard naive DDIM suffers from an 8.74% bit error rate because small recovery errors accumulate. A shorter direct method avoids many of those errors, but it also limits the

TABLE III
COMPARISON OF INVERSION METHODS ACROSS 50 STEPS.

Inversion Method	Bit Error Rate	Visual PSNR	Speed
Naive DDIM Inversion (as in [5])	8.742%	32.14 dB	0.58 s
Direct x_0 Shortcut (10 steps)	0.000%	50.79 dB	0.14 s
Our Fixed-Point DDIM (50 steps)	0.000%	45.59 dB	0.63 s

image-generation process. The proposed repeated-correction method uses the full 50 steps while maintaining a **0.000% Bit Error Rate** and sub-second execution in the reported tests.

V. DISCUSSION, LIMITATIONS, AND FUTURE WORK

A. What the Results Show

The experiments show that the three parts of the system support one another. Repeated correction improves the recovered noise, natural-looking noise encoding reduces obvious statistical patterns, and error correction protects the final message. The automatic header also makes the receiver easier to use because important settings travel with the payload instead of being entered manually.

The results should be understood within the tested setup. A zero bit error rate across 20 trials shows consistent performance in those trials, but it does not prove that every possible image or communication channel will produce the same result. Similarly, passing the reported statistical tests means that those tests did not find a clear difference in the evaluated samples. It does not guarantee that every future steganalysis method will fail.

B. Current Limitations

The present evaluation uses small 64×64 images and a controlled diffusion pipeline. Larger images may require more memory, longer processing time, and additional testing. The sender and receiver must also use compatible software, model weights, and secret settings. Even a small implementation difference can affect the recovered internal values.

Another limitation is channel modification. Social-media platforms often resize images, change color values, remove metadata, or apply lossy compression. Mild changes may be repaired by the error-correction layer, but strong changes can destroy too much hidden information. Cropping is especially difficult because it removes part of the generated carrier rather than merely adding small noise.

The system also carries relatively short messages compared with ordinary communication tools. This is intentional: increasing the payload too aggressively can reduce reliability or create detectable patterns. The method is therefore better suited to short text, identifiers, verification codes, and compact metadata than to large files.

C. Responsible Use

Steganography can support privacy, ownership tracking, and secure research communication, but it can also be misused. Practical deployments should follow applicable laws, organizational policies, and consent requirements. Systems should include access controls and should not be presented as a replacement for encryption. Encryption protects message contents, while

steganography mainly attempts to hide the presence of the message; using both provides a stronger design.

D. Future Improvements

Future work can evaluate larger and more varied images, additional diffusion models, and a greater number of test messages. It would also be useful to test common real-world changes such as JPEG compression, resizing, screenshots, color adjustment, and social-media upload pipelines. Adaptive protection could automatically select a stronger error-correction level when the expected channel is unreliable.

Further research may also combine the method with modern encryption and key-management systems, reduce the time needed for repeated recovery, and develop clearer capacity guidelines. These improvements would help determine how well the approach moves from a controlled experiment to practical communication environments.

VI. CONCLUSION

In this paper, we introduced an exact, distribution-preserving, and error-resilient generative diffusion steganography pipeline. By pairing an iterative fixed-point DDIM inversion solver with Distribution-Preserving Arithmetic Coding (DPAC) and rate-adaptive systematic error correction, our system eliminates trajectory drift, shows no statistically significant deviation from Gaussian noise in our tests, and provides autonomous error recovery. Rigorous benchmarks across 20 trials demonstrated a flawless 0.000000% Bit Error Rate, 100% extraction success, high visual fidelity (PSNR ≥ 45.59 dB), and no detectable statistical deviation from the expected noise distribution ($p = 0.4191$).

REFERENCES

- [1] V. Holub, J. Fridrich, and T. Denemark, "Universal distortion function for steganography in an arbitrary domain," *EURASIP J. Inf. Secur.*, vol. 2014, no. 1, pp. 1–13, 2014.
- [2] V. Holub and J. Fridrich, "Designing steganographic distortion using directional filters," in *Proc. IEEE Int. Workshop Inf. Forensics Secur. (WIFS)*, 2012, pp. 234–239.
- [3] M. Boroumand, M. Chen, and J. Fridrich, "Deep residual network for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 5, pp. 1181–1193, 2019.
- [4] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.
- [5] Q. Zhou, P. Wei, Z. Qian, X. Zhang, and S. Li, "Improved generative steganography based on diffusion model," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 7, pp. 6494–6507, Jul. 2025.
- [6] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 6840–6851.
- [7] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [8] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 586–595.